



Nombre y Apellidos:
Sección:

RUT:
Firma:

CC1002 – 2025 Semestre Primavera - Control 2 - Duración: 2:00 horas

Indicaciones Generales:

- 1.- Lea atentamente las preguntas. **Habrá dos rondas de preguntas de enunciado durante el control.**
- 2.- Debe responder en las hojas de respuestas que se le entregarán – no se aceptarán hojas adicionales.
- 3.- Siempre debe incluir la receta de diseño al escribir una función, implementando al menos un test.

Pregunta 1 (50%)

Una baraja de naipes francés se compone de cartas, donde cada una tiene una **pinta** (Corazón, Diamante, Trébol o Pica) y un **valor** (A, 2, 3, ..., 9, 10, J, Q, K). Al respecto, una Carta puede ser representada por la siguiente estructura:

```
# Carta: pinta(str) valor(int)
estructura.crear("Carta", "pinta valor")
```

Por simplicidad se representa el valor como un número entero entre 1 y 13 (A=1, J=11, Q=12, K=13), y la pinta por la letra inicial de cada una ('C', 'D', 'T', 'P'). No se consideran naipes conocidos como "Comodín" o "Joker".

Una **mano de cartas** puede ser representada como una lista de estructuras Carta. No existe un largo mínimo ni máximo para la lista. Por ejemplo, las cartas: 8 de Corazones, Rey de Diamante y As de Pica, se pueden representar con la siguiente lista:

```
L1 = lista(Carta('C', 8), lista(Carta('D', 13), lista(Carta('P', 1), listaVacía)))
```

En una mano de cartas pueden existir las siguientes combinaciones de cartas:

- **Par:** Dos cartas consecutivas con el mismo valor, sin importar la pinta. En el caso que una misma carta sirva para formar más de un par con otra carta, solo se considera una combinación, es decir, si tengo 3 cartas de igual valor solo hay un “par”, si tengo 4 cartas de igual valor son dos “pares”.
- **Escalera:** La mano completa debe ser una secuencia de 3 o más valores consecutivos en orden creciente de la misma pinta. En el caso del Rey (K=13) se considera que su continuación es el As (A=1), y se le denomina Escalera cíclica.

Al respecto, se necesita definir las siguientes funciones.

Indicaciones:

- Suponga que existe la función **largo(L)**, que devuelve la cantidad de cartas en la lista L.
- Para los ejemplos de las funciones, suponga la existencia de las siguientes listas de cartas:

```
LC1 = lista(Carta('P', 2), lista(Carta('D', 2), lista(Carta('T', 2), lista(Carta('T', 5), lista(Carta('D', 5), listaVacía)))))
LC2 = lista(Carta('P', 2), lista(Carta('T', 5), lista(Carta('D', 2), lista(Carta('D', 5), lista(Carta('T', 2), listaVacía)))))
LC3 = lista(Carta('T', 12), lista(Carta('T', 13), lista(Carta('T', 1), lista(Carta('T', 2), listaVacía)))))
LC4 = lista(Carta('T', 13), lista(Carta('T', 1), lista(Carta('T', 2), lista(Carta('T', 12), listaVacía)))))
LC5 = lista(Carta('D', 11), lista(Carta('P', 12), lista(Carta('C', 13), listaVacía)))))
```

(A) (2.2p) Escriba la función recursiva **pares(L)**, que recibe una **lista de Cartas**. La función entrega una lista con las cartas de la lista que forman pares, siguiendo el mismo orden de la lista original. Los pares solo se pueden formar entre dos cartas consecutivas en la lista. Ejemplos:

- **pares(LC1)** entrega:
`lista(Carta('P', 2), lista(Carta('D', 2), lista(Carta('T', 5), lista(Carta('D', 5), listaVacía)))))`
- **pares(LC2)** entrega: `listaVacía`

(B) (2.2p) Escriba la función recursiva **consecutivos(L)**, que recibe una **lista de Cartas**. La función indica si **todas** las cartas en la lista son valores consecutivos de la misma pinta, siguiendo el mismo orden de la lista original. Ejemplos:

- **consecutivos(LC3)** entrega: `True`
- **consecutivos(LC4)** entrega: `False`
- **consecutivos(LC5)** entrega: `False`

(C) (1.6p) Escriba la función **puntaje(L)**, que recibe una **lista de Cartas**, y asigna una puntuación a las Cartas en la lista (en el orden que vienen), de acuerdo con el siguiente criterio:

- Cada par de cartas aporta 3 puntos.
- Una escalera (ver definición arriba) aporta 2 puntos multiplicado por el largo de la escalera.

Ejemplos:

- **puntaje(LC1)** entrega: 6
- **puntaje(LC3)** entrega: 8



Nombre y Apellidos:
Sección:

RUT:
Firma:

Solución Pregunta 1:

(A) Asignación de puntajes:

- **(0.1p)** Contrato
- **(0.1p)** Propósito + Ejemplo
- **(0.1p)** Encabezado función
- **(0.1p)** Testing
- Cuerpo función:
 - **(0.3p)** Caso Base: lista vacía
 - **(0.3p)** Caso Base: 1 carta en la lista
 - Caso Recursivo:
 - **(0.3p)** Obtener la carta actual y la siguiente
 - **(0.3p)** Ver si coinciden sus valores
 - **(0.4p)** Incluir ambas cartas si coinciden
 - **(0.2p)** Continuar la recursión

Implementación de ejemplo:

```
# previa
from lista import *
from absfun import *

# Carta: pinta(str) valor(int)
estructura.crear("Carta", "pinta valor")

L1 = lista(Carta('C', 8), lista(Carta('D', 13), lista(Carta('P', 1), listaVacía)))

LC1 = lista(Carta('C',10), lista(Carta('D',11), lista(Carta('T',12), \
    lista(Carta('T',13), lista(Carta('P', 1), listaVacía)))))
LC2 = lista(Carta('D',12), lista(Carta('D',13), lista(Carta('D', 1), \
    lista(Carta('D', 2), listaVacía ))))
LC3 = lista(Carta('P', 4), lista(Carta('D', 4), lista(Carta('T', 4), \
    lista(Carta('P', 7), lista(Carta('D', 7), listaVacía)))))
LC4 = lista(Carta('T', 3), lista(Carta('T', 4), lista(Carta('T', 6), listaVacía)))
```



Nombre y Apellidos:
Sección:

RUT:
Firma:

```
# pares: lista(Carta) -> lista(Carta)
# entrega una lista con las cartas que forman pares
# ej: pares(LC3) entrega: lista(Carta('P', 4), lista(Carta('D', 4),
#                               lista(Carta('P', 7), lista(Carta('D', 7), listaVacía))))
def pares(L):
    assert esLista(L)

    # caso base 1: listaVacía
    if L == listaVacía:
        return listaVacía

    # caso base 2: última carta en la lista
    # no hay más cartas para intentar formar pares
    if cola(L) == listaVacía:
        return listaVacía

    # caso recursivo
    Cactual = cabeza(L)
    Csiguiente = cabeza(cola(L))

    # si coinciden, conservo ambas cartas, y continúo más adelante
    if Cactual.valor == Csiguiente.valor:
        return lista(Cactual, lista(Csiguiente, pares(cola(cola(L)))))
    else:
        # si no, entonces continuo en la siguiente carta
        return pares(cola(L))

# test
assert pares(LC3) == lista(Carta('P',4),lista(Carta('D',4),\
        lista(Carta('P',7),lista(Carta('D',7), listaVacía))))
assert pares(LC1) == listaVacía
```



Nombre y Apellidos:
Sección:

RUT:
Firma:

(B) Asignación de puntajes:

- **(0.1p)** Contrato
- **(0.1p)** Propósito + Ejemplo
- **(0.1p)** Encabezado función
- **(0.1p)** Testing
- Cuerpo función:
 - **(0.3p)** Caso Base: lista vacía
 - **(0.3p)** Caso Base: 1 carta en la lista
 - Caso Recursivo:
 - **(0.3p)** Obtener la carta actual y la siguiente
 - **(0.3p)** Descartar si sus pintas no coinciden
 - **(0.2p)** Ver si son cartas consecutivas
 - **(0.2p)** Ver el caso K -> A
 - **(0.2p)** Continuar la recursión

Implementación de ejemplo:

```
# consecutivos: lista(Carta) -> bool
# indica si todas las cartas en la lista forman una escala consecutiva de valores
# ej: consecutivos(LC2) entrega True
def consecutivos(L):
    assert esLista(L)

    # caso base 1: listaVacía
    if L == listaVacía:
        return True

    # caso base 2: última carta en la lista
    if cola(L) == listaVacía:
        return True

    # caso recursivo
    Cactual = cabeza(L)
    Csiguiente = cabeza(cola(L))
    # descartamos si es que no tienen la misma pinta
    if Cactual.pinta != Csiguiente.pinta:
        return False

    # revisamos si la carta actual y la siguiente son consecutivas
    if Cactual.valor + 1 == Csiguiente.valor:
        return consecutivos(cola(L))

    # o bien, que la actual sea K y la siguiente A
    elif Cactual.valor == 13 and Csiguiente.valor == 1:
        return consecutivos(cola(L))

    else:
        return False

# test
assert consecutivos(LC1) == False
assert consecutivos(LC2) == True
assert consecutivos(LC4) == False
```



Nombre y Apellidos:
Sección:

RUT:
Firma:

(C) Asignación de puntajes:

- **(0.1p)** Contrato
- **(0.1p)** Propósito + Ejemplo
- **(0.1p)** Encabezado función
- **(0.1p)** Testing
- Cuerpo función:
 - **(0.5p)** Calcular puntaje Pares
 - **(0.7p)** Calcular puntaje escalera (revisando que la cadena sea mayor o igual a 3)

Implementación de ejemplo:

```
# puntaje: lista(Carta) -> int
# de acuerdo a las cartas, asigna un puntaje
# ej: puntaje(LC2) == 8
def puntaje(L):
    assert esLista(L)

    p = 0

    # puntaje pares
    Lpares = pares(L)
    p = p + largo(Lpares)//2 * 3

    # puntaje escalera
    if consecutivos(L) and largo(L) >= 3:
        p = p + 2 * largo(L)

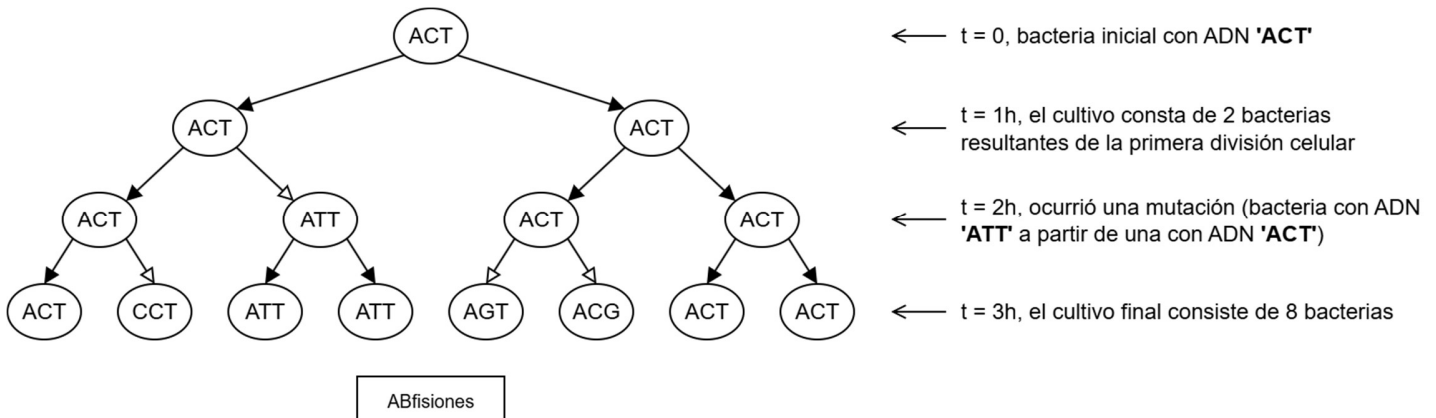
    return p

# test
assert puntaje(LC2) == 8
assert puntaje(LC3) == 6
```

Pregunta 2 (50%)

Las **bacterias de Choufeb** son microorganismos que realizan su proceso de división celular cada **1 hora**. Haciendo uso de un árbol binario (AB), se puede estudiar el crecimiento de un cultivo bacteriano a lo largo del tiempo, originado a partir de una bacteria de Choufeb inicial.

En este árbol, cada nodo contiene el ADN de una bacteria representado como un **string**. Cuando una bacteria se divide, se crean siempre **dos nodos hijos**, con el mismo ADN del padre, **excepto cuando ocurre una MUTACIÓN** (cambia su ADN) durante la división. Un ejemplo de esto es el AB **ABfisiones**:



Las flechas en blanco indican **instancias de mutación**, en las que el ADN de una bacteria no se replicó correctamente (el valor de un nodo es distinto al de uno de sus hijos).

Considere la siguiente definición de árbol binario:

```
# AB: valor(str) izq(AB) der(AB)
estructura.crear('AB', 'valor izq der')
```

Para los ejemplos de las funciones, suponga que existe el árbol **ABfisiones**. Al respecto, escriba las siguientes funciones recursivas:

(A) (3.0p) Escriba la función recursiva **mutaciones(A)**, que recibe un AB con ADNs de bacterias, y entrega la **cantidad de instancias de mutación** ocurridas en total (en la figura, corresponde a la cantidad de flechas en blanco). Ejemplo:

- **mutaciones(ABfisiones)** entrega: 4

(B) (3.0p) Escriba la función recursiva **cultivo(A, t)**, que recibe un AB con ADNs de bacterias y un número entero de horas. La función entrega una lista con los ADNs de las bacterias en el cultivo, luego de que haya transcurrido el tiempo indicado. Ejemplos:

- **cultivo(ABfisiones, 0)** entrega: lista('ACT', listaVacia)
- **cultivo(ABfisiones, 2)** entrega: lista('ACT', lista('ATT', lista('ACT', lista('ACT', listaVacia))))
- **cultivo(ABfisiones, 3)** entrega: lista('ACT', lista('CCT', lista('ATT', lista('ATT', lista('AGT', lista('ACG', lista('ACT', lista('ACT', listaVacia))))))))

Indicaciones:

- Como indica el enunciado, puede asumir que los nodos internos de los AB de entrada tendrán siempre 2 hijos.
- Para la parte (B), puede usar la función **concatenar(L1, L2)**, que entrega una nueva lista con todos los valores de la lista L1 y a continuación todos los valores de la lista L2.
- Para la parte (B), puede asumir que **t** nunca será mayor al tiempo de vida del cultivo, y puede omitir el caso base cuando A es vacío.



Nombre y Apellidos:
Sección:

RUT:
Firma:

Solución Pregunta 2:

(A) Asignación de puntajes:

- **(0.2p)** Contrato
- **(0.1p)** Propósito + Ejemplo
- **(0.1p)** Encabezado función
- **(0.2p)** Testing
- Cuerpo función:
 - **(0.1p)** Caso de Borde: Árbol vacío
 - **(0.2p)** Caso Base: Nodo hoja
 - Caso Recursivo
 - **(0.7p)** Da el resultado recursivo correcto cuando los dos nodos hijos son distintos
 - **(0.7p)** Da el resultado recursivo correcto cuando un solo nodo hijo es distinto
 - **(0.7p)** Da el resultado recursivo correcto cuando ningún nodo hijo es distinto

Implementación de ejemplo:

```
# mutaciones: AB(str) -> int
# Entrega la cantidad de mutaciones que
# ocurrieron dentro del árbol dado.
# Ejemplo: mutaciones(ABfisiones) -> 4
def mutaciones(A):
    if vacio(A):
        return 0
    if vacio(A.izq) and vacio(A.der):
        return 0
    if A.valor != A.izq.valor and A.valor != A.der.valor:
        return 2 + mutaciones(A.izq) + mutaciones(A.der)
    elif A.valor != A.izq.valor or A.valor != A.der.valor:
        return 1 + mutaciones(A.izq) + mutaciones(A.der)
    else:
        return mutaciones(A.izq) + mutaciones(A.der)

# Tests
assert mutaciones(fisiones) == 4
assert mutaciones(fisiones.izq) == 2
assert mutaciones(fisiones.der) == 2
```



Nombre y Apellidos:
Sección:

RUT:
Firma:

(B) Asignación de puntajes:

- **(0.2p)** Contrato
- **(0.1p)** Propósito + Ejemplo
- **(0.1p)** Encabezado función
- **(0.2p)** Testing
- Cuerpo función:
 - **(0.6p)** Caso Base: $t == 0$
 - Caso Recursivo
 - **(0.6p)** Obtiene las bacterias en el cultivo luego de $t - 1$ horas del árbol izquierdo
 - **(0.6p)** Obtiene las bacterias en el cultivo luego de $t - 1$ horas del árbol derecho
 - **(0.6p)** Retorna como resultado final la unión de ambas listas

Implementación de ejemplo:

```
# cultivo: AB(str) int -> lista(str)
# Entrega una lista con los ADN's de las bacterias
# en el cultivo luego del tiempo indicado.
# Ejemplo: cultivo(ABfisiones, 0) -> lista('ACT', listaVacía)
def cultivo(A, t):
    if t == 0:
        return lista(A.valor, listaVacía)
    else:
        listaIzq = cultivo(A.izq, t - 1)
        listaDer = cultivo(A.der, t - 1)
        return concatenar(listaIzq, listaDer)

# Tests
assert cultivo(fisiones, 0) == lista('ACT', listaVacía)
assert cultivo(fisiones, 2) == lista('ACT', lista('ATT', lista('ACT', lista('ACT', listaVacía))))
assert cultivo(fisiones, 3) == lista('ACT', lista('CCT', lista('ATT', lista('ATT', lista('AGT',
lista('ACG', lista('ACT', lista('ACT', listaVacía))))))))))
```